

6.1 利用networkx创建图

6.1.1 利用networkx创建无向图

使用 networkx 库来创建和分析无向图是一个直观且强大的方法。networkx 是一个Python包，用于创建、操作复杂网络的结构、动态和功能。以下是如何使用 networkx 来创建无向图并进行简单分析的基本步骤。

安装 networkx

如果你还没有安装 networkx，你可以通过pip来安装它：

```
pip install networkx
```

创建无向图

在 networkx 中，你可以使用 Graph 类来创建无向图。

```
import networkx as nx

# 创建一个空的无向图
G = nx.Graph()

# 添加节点
G.add_node(1)
G.add_nodes_from([2, 3, 4])

# 添加边
G.add_edge(1, 2)
G.add_edges_from([(2, 3), (3, 4), (4, 1)])

# 查看图
print(G.nodes()) # 查看节点
print(G.edges()) # 查看边
```

简单分析

1. 图的度 (Degree)

节点的度是指与该节点连接的边的数量。在无向图中，每个连接算作一条边。

```
print(G.degree()) # 查看所有节点的度
for node, degree in G.degree():
    print(f"Node {node} has degree {degree}")
```

2. 图的邻接节点

你可以获取某个节点的所有邻接节点。

```
print(list(G.neighbors(1))) # 获取节点1的所有邻接节点
```

3. 图的连通性

检查图是否是连通的（即图中任意两个节点之间都存在路径）。

```
print(nx.is_connected(G)) # 对于无向图，检查图是否连通
```

4. 图的遍历

你可以使用深度优先搜索 (DFS) 或广度优先搜索 (BFS) 来遍历图。

```
# 深度优先搜索
print(list(nx.dfs_preorder_nodes(G, source=1)))
```

```
# 广度优先搜索
print(list(nx.bfs_edges(G, source=1)))
```

5. 图的绘图

虽然绘图不是直接的分析步骤，但它对于理解图的结构非常有帮助。

```
import matplotlib.pyplot as plt

# 使用matplotlib绘制图
nx.draw(G, with_labels=True, font_weight='bold')
plt.show()
```

6.1.2 利用networkx创建有向图

在 networkx 中，创建有向图并分析它与无向图非常相似，但你需要使用 DiGraph 类而不是 Graph 类。以下是如何使用 networkx 来创建有向图并进行简单分析的基本步骤。

创建有向图

```
import networkx as nx

# 创建一个空的有向图
DG = nx.DiGraph()

# 添加节点
DG.add_node(1)
DG.add_nodes_from([2, 3, 4])

# 添加有向边
DG.add_edge(1, 2)
DG.add_edges_from([(2, 3), (3, 4), (4, 1)])

# 查看图
print(DG.nodes()) # 查看节点
print(DG.edges()) # 查看有向边
```

简单分析

1. 图的入度和出度

对于有向图，每个节点都有入度（指向该节点的边的数量）和出度（从该节点出发的边的数量）。

```
print(DG.in_degree()) # 查看所有节点的入度
print(DG.out_degree()) # 查看所有节点的出度

for node, in_deg, out_deg in zip(DG.nodes(), DG.in_degree(), DG.out_degree()):
    print(f"Node {node} has in-degree {in_deg} and out-degree {out_deg}")
```

2. 图的邻接节点

对于有向图，你需要区分前驱节点（指向当前节点的节点）和后继节点（从当前节点出发指向的节点）。

```
print(list(DG.predecessors(2))) # 获取节点2的所有前驱节点
print(list(DG.successors(2))) # 获取节点2的所有后继节点
```

3. 图的连通性

对于有向图，连通性的概念更加复杂，因为你需要区分强连通（任意两个节点都互相可达）和弱连通（忽略边的方向后任意两个节点都可达）。

```
print(nx.is_strongly_connected(DG)) # 检查图是否是强连通的
print(nx.is_weakly_connected(DG)) # 检查图是否是弱连通的（忽略方向）
```

4. 图的遍历

有向图的遍历同样可以使用深度优先搜索（DFS）和广度优先搜索（BFS），但结果会根据边的方向而变化。

```
# 深度优先搜索
print(list(nx.dfs_preorder_nodes(DG, source=1)))

# 广度优先搜索
print(list(nx.bfs_edges(DG, source=1)))
```

5. 图的绘图

绘图时，`networkx` 会自动根据是否有向来绘制箭头，表示边的方向。

```
import matplotlib.pyplot as plt
```

```
# 使用matplotlib绘制有向图
```

```
nx.draw(DG, with_labels=True, font_weight='bold', arrowstyle='->', arrowsize=20)
```

```
plt.show()
```

通过以上步骤，你可以开始使用 `networkx` 来创建和分析有向图了。`networkx` 的丰富功能允许你进行更深入的图论分析，包括最短路径计算、网络流、社区检测等。